

Fundamentos del *testing*

Fundamentos del *testing*

Definiciones de testing según IEEE

“Es el proceso de operar un sistema o componente en determinadas condiciones, observando y registrando los resultados elaborando una evaluación y devolución de los mismos”.

Conceptos de validación y verificación

Verificación: “¿Estoy construyendo correctamente el producto?”

Validación: “¿Estoy construyendo el producto correcto?”

- Relación componentes del *testing*. *Testing* a través del ciclo de vida del software.
- Niveles de *testing* modelo-V.

Los fundamentos de las pruebas de software

Según algunas opiniones una proporción elevada de los sistemas informáticos no cumplen con las expectativas por culpa de software insuficientes las pruebas antes de su liberación.

Las pruebas de software no pueden garantizar que el software no tiene fallo, pero si minimizar el riesgo de fallos en desarrollo una vez que el software está en uso.

La prueba de software es un método para identificar las fallas en el producto antes de ser liberada, lo que incluye los defectos en el código de software y su ejecución, así como toda diferencia de potencial entre lo que el código se supone que debe hacer y lo que realmente hace.

Las pruebas de software no es simplemente un caso de examen de la documentación o ejecutar un programa de software para verificar si hay errores.

Es importante para la prueba de software la selección de la persona adecuada (s) para realizar las pruebas, y comunicar los resultados de las pruebas para los desarrolladores, es

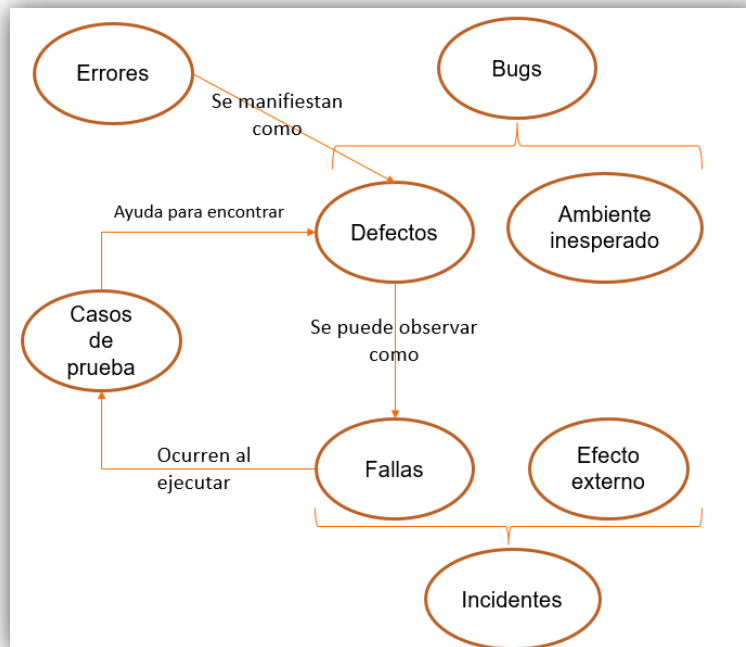
decir que se requiere una comprensión básica de la psicología de los probadores y desarrolladores.

Además, es importante considerar cómo probadores pueden comunicar los resultados de la prueba para los desarrolladores.

Axiomas del testing

1. El software siempre se testea.
2. Un buen caso de testeo es el que muestra que el programa no anda, no que funciona correctamente.
3. Nada es más difícil que saber cuándo dejar de testear.
4. Uno no puede autotestearse.
5. Parte ineludible del test es determinar cuál es el resultado esperado.
6. El test debe ser reproducible.
7. Casos para condiciones válidas e inválidas.
8. Inspeccionar todos los resultados de lo testeado.
9. A medida que avanza el testeo crece la probabilidad de no encontrar errores.
10. El mejor programador debe realizar el testeo de caja blanca.
11. El mejor usuario debe realizar el testeo de caja negra.
12. El testeo es parte del proceso de desarrollo.
13. Nunca se modifica un programa a efectos de ser más fácilmente testeado.

Errores, defectos y fallas



Objetivos de V&V

Descubrir defectos (para corregirlos)

- Provocar fallas (una forma de detectar defectos).
- Revisar los productos (otra forma de detectar defectos).

Evaluar la calidad de los productos

- El probar o revisar el software da una idea de la calidad del mismo.

Identificación y corrección de defectos

Identificación de defectos

- Es el proceso de determinar que defecto o defectos causaron la falla.

Corrección de defectos

- Es el proceso de cambiar el sistema para remover los defectos.

Ejemplo

El programa lee tres números enteros, los que son interpretados como representaciones de las longitudes de los lados de un triángulo.

El programa escribe un mensaje que informa si el triángulo es escaleno, isósceles o equilátero.

Quiero detectar defectos probando (testeando) el programa.

Posibles casos a probar:

- lado1 = 0, lado2 = 1, lado3 = 0 Resultado = error
- lado1 = 2, lado2 = 2, lado3 = 3 Resultado = isósceles

Estos son casos de prueba:

Comparo el resultado esperado con el obtenido:

- Si son distintos probablemente haya fallado el programa.

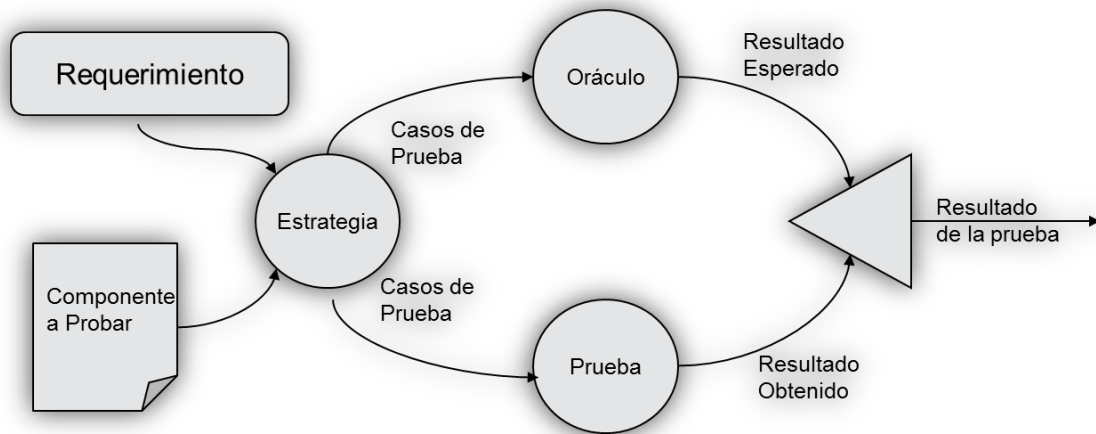
Intuitivamente que otros casos sería bueno probar:

- lado1 = 2, lado2 = 3, lado3 = 4 Resultado = escaleno
- lado1 = 2, lado2 = 2, lado3 = 2 Resultado = equilátero
- ¿Por qué estos casos?
 - Al menos probé un caso para cada respuesta posible del programa (error, escaleno, isósceles, equilátero).

Otra forma para detectar defectos es revisar el código:

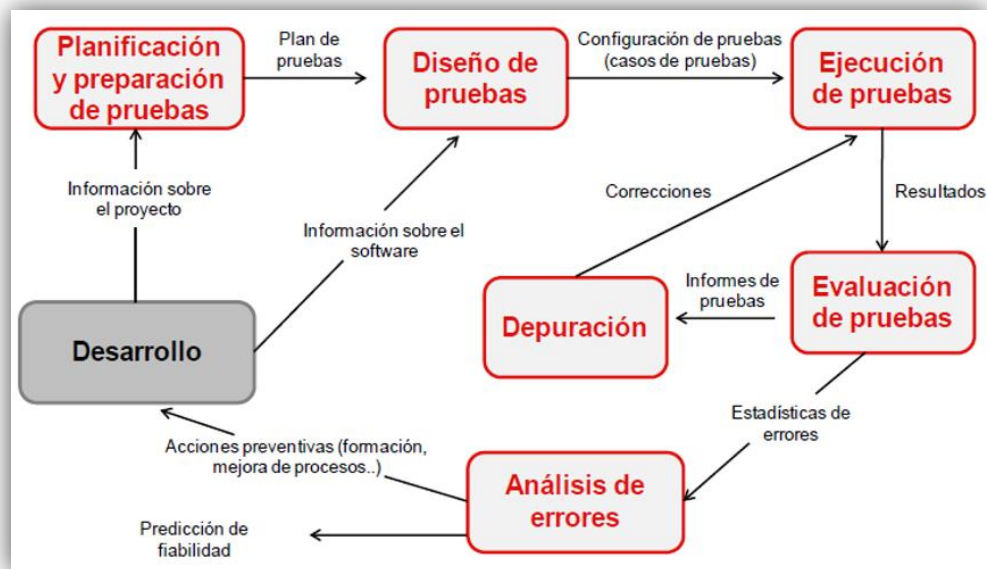
- *If l1 = l2 or l2 = l3 then*
write ("equilátero")
else ...
- En lugar del *or* me doy cuenta que debería ir un *and*
- Se puede revisar solo
- Se puede revisar en grupos

El proceso de la prueba

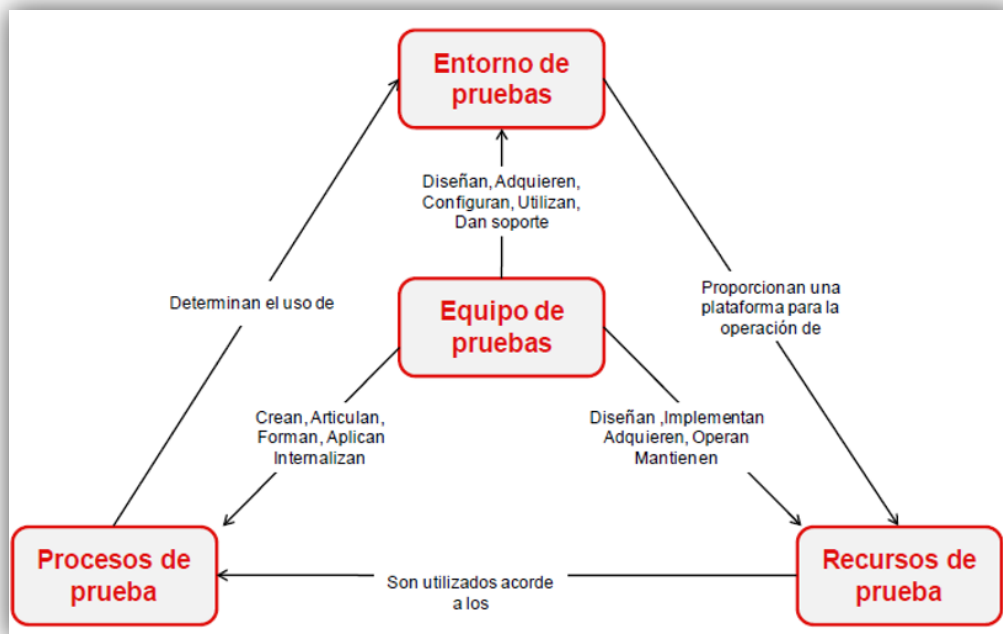


Las pruebas siempre deben realizarse contra un resultado esperado.

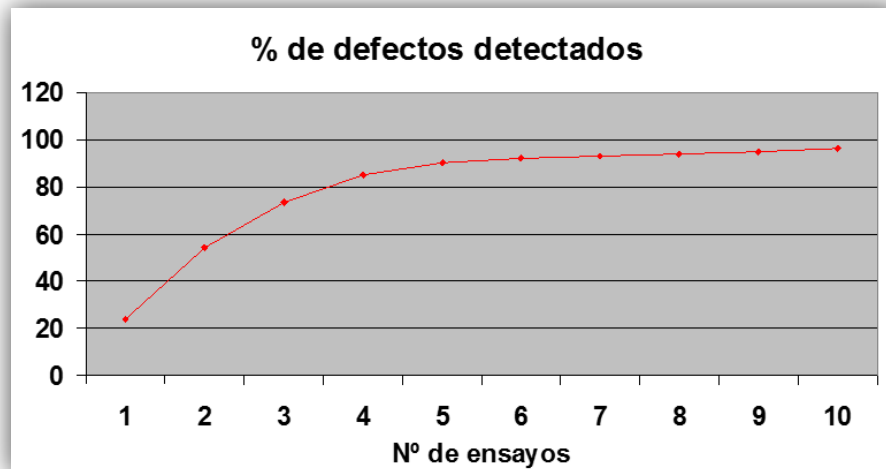
Los resultados deben ser revisados minuciosamente.



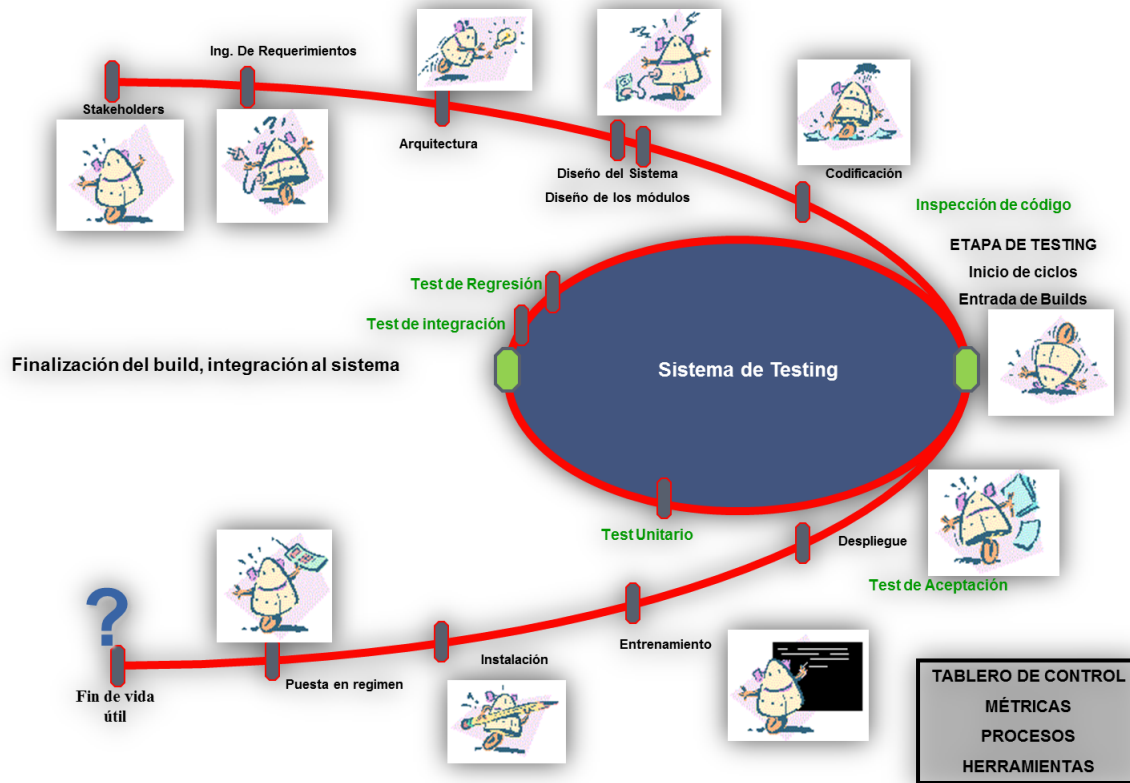
Relación componentes sistema de pruebas



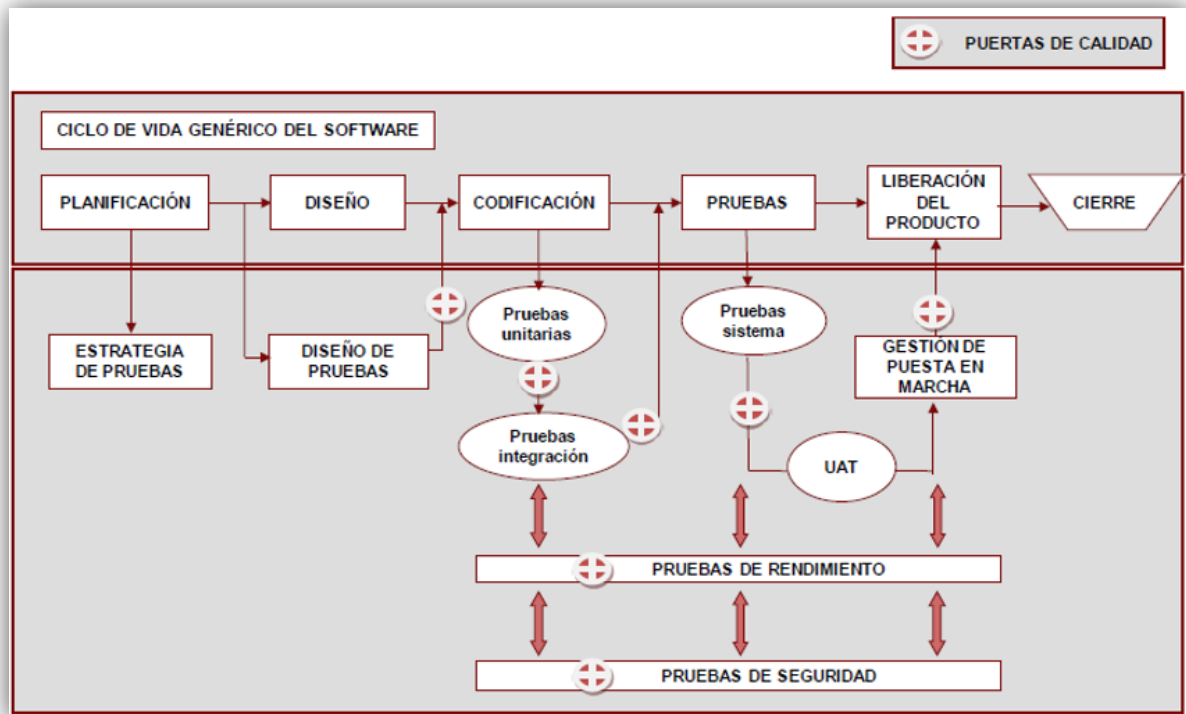
Defectos encontrados versus pruebas



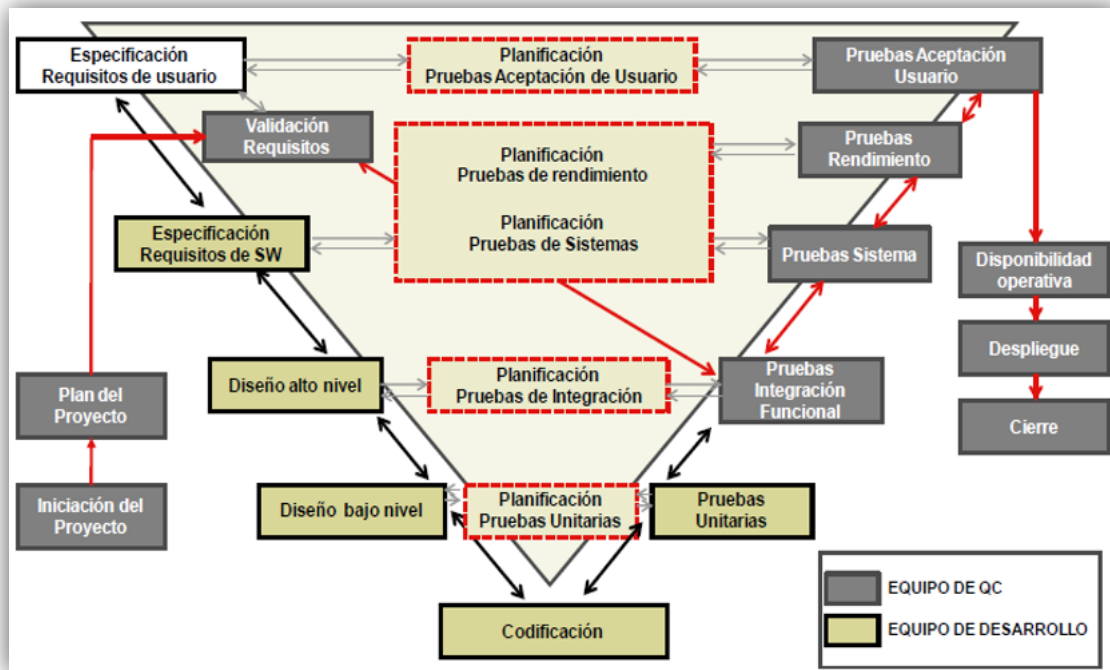
Ciclo de vida genérico



Ciclo de vida



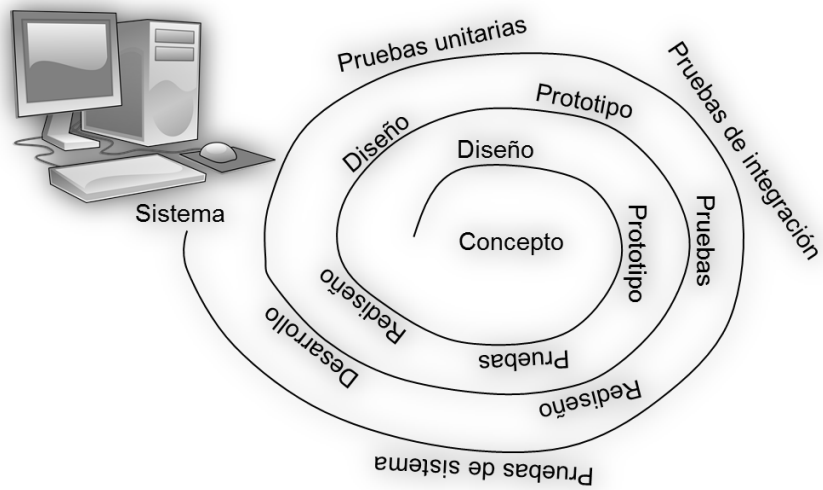
Modelo V



Modelo prototipo



Modelo espiral



Modelo incremental

